INSTITUTO SAN IGNACIO DE LOYOLA – ISIL

PROCESO DE APRENDIZAJE 3

MÓDULO DE CONSULTA Y SUPERVISIÓN DE PEDIDOS

Desarrollo de Aplicaciones Empresariales Avanzado · 2026-20

Estudiante	Matías Farro Ortiz
NRC	_____
Docente	_____
Periodo	2026-20
Proyecto base	Sistema de Pedidos – Spring Boot

Informe técnico de evolución del proyecto base suministrado por el docente.

1. Objetivo y alcance

El objetivo de esta actividad es evolucionar el Sistema de Pedidos suministrado por el docente e implementar un módulo de consulta y supervisión que integre Spring MVC con vistas JSP, respuestas JSON, Spring Security y consultas explícitas con Spring JDBC. La solución conserva Spring Data JPA para el registro transaccional de pedidos y mantiene la integración HTTP existente con inventario-service.

El alcance de PA3 no consiste en rehacer la edición/eliminación de PA1 ni el flujo de anulación de PA2. Se centra en consultar, consolidar, proteger y probar la información ya almacenada. Las consultas nuevas se implementan mediante SQL parametrizado y no mediante métodos derivados de JpaRepository ni JPQL.

2. Arquitectura general

El recorrido de consulta comienza en el navegador. Spring Security valida la sesión, los roles y la protección CSRF correspondiente. DispatcherServlet dirige la solicitud al controlador MVC o al controlador REST. Ambos delegan en PedidoReporteService, que aplica las validaciones y llama a PedidoReporteJdbcRepository. Este repositorio ejecuta consultas parametrizadas sobre la tabla pedido mediante el mismo DataSource utilizado por JPA.

La operación de registro conserva el flujo anterior: PedidoService coordina PedidoRepository (Spring Data JPA) y el cliente HTTP de inventario. La indisponibilidad del servicio externo no impide consultar los pedidos que ya están guardados en la base local.

Diagrama actualizado: **docs/arquitectura-pa3.puml**. El archivo muestra Navegador, Spring Security, DispatcherServlet, controladores JSP/REST, Service, repositorios JPA/JDBC, MySQL e inventario-service.

3. Cambios realizados por capa

Repositorio JDBC

Se agregó **buscarPedidosPorClienteYTotalMinimo(cliente, totalMinimo)**. La consulta usa NamedParameterJdbcTemplate, parámetros nombrados, reutiliza PedidoRowMapper y ordena los resultados por id descendente. El texto del cliente y el monto mínimo se envían como valores; no se concatenan en el SQL.

También se agregó **listarClientesFrecuentes(minPedidos)**, con GROUP BY cliente y HAVING COUNT(*) >= :minPedidos. La consulta produce una proyección con cliente, totalPedidos, totalUnidades, importeAcumulado y ticketPromedio.

Capa Service

PedidoReporteService y PedidoReporteServiceImpl exponen las dos operaciones. La capa valida que el cliente no sea nulo ni vacío, que totalMinimo sea mayor o igual a cero y que minPedidos sea al menos uno. Las reglas no se trasladan al repositorio.

Capa web y API

Se incorporó PedidoReporteWebController con @Controller, @GetMapping y Model para cargar la vista /WEB-INF/views/reportes-pedidos.jsp. La vista presenta el resumen existente, el formulario de búsqueda avanzada y la tabla de clientes frecuentes. Los endpoints JSON se mantienen en PedidoReporteController y reutilizan el mismo Service.

Seguridad

SecurityConfig declara el acceso GET a /reportes/** y a los endpoints de reportes para OPERADOR y CONSULTA. POST /pedidos continúa siendo exclusivo de OPERADOR. No se deshabilitó CSRF; el formulario de pedidos y el cierre de sesión conservan sus tokens.

4. Casos funcionales obligatorios

Los cinco escenarios se deben ejecutar sobre la misma solución. La tabla resume el recorrido técnico y el resultado que debe comprobarse en el entorno local.

Caso	Recorrido / comprobación
1. Búsqueda avanzada	Registrar al menos tres pedidos. Consultar por cliente y total mínimo tanto en JSP como en GET /api/v1/pedidos/busqueda-avanzada. Comparar IDs y totales con MySQL.
2. Clientes frecuentes	Registrar pedidos para al menos dos clientes. Consultar minPedidos=2 y comprobar totalPedidos, totalUnidades, importeAcumulado y ticketPromedio.
3. Parámetros inválidos	Enviar cliente vacío, totalMinimo negativo y minPedidos menor que uno. La API debe responder 400 y la JSP debe presentar un mensaje controlado.
4. Inventario no disponible	Detener inventario-service. Un nuevo registro debe informar indisponibilidad (503); los reportes locales deben seguir funcionando porque consultan MySQL.
5. Seguridad y CSRF	Anónimo: redirección al login. CONSULTA: puede consultar. CONSULTA con CSRF: no puede registrar. OPERADOR sin CSRF: POST rechazado.

Las capturas y resultados de ejecución deben generarse en el equipo del estudiante. La carpeta docs/evidencias incluye una guía con los nombres y contenidos sugeridos para cada captura. No se incorporan capturas simuladas ni se afirma que una operación externa se haya ejecutado sin comprobarla.

Ejemplos de endpoints

GET /sistema-pedidos/api/v1/pedidos/busqueda-avanzada?cliente=Ana&totalMinimo=500
GET /sistema-pedidos/api/v1/pedidos/clientes-frecuentes?minPedidos=2
GET /sistema-pedidos/reportes/pedidos

5. Pruebas automatizadas

Se incorporó la clase Pa3ReportesFeatureTest con ocho pruebas nuevas para las funcionalidades de PA3. Las pruebas de integración utilizan el perfil test y la base H2 en memoria, en modo de compatibilidad MySQL. La ejecución no requiere una instancia externa de MySQL para estas pruebas.

Las pruebas cubren: (1) búsqueda JDBC por cliente y monto mínimo; (2) agrupación de clientes frecuentes y cálculo de indicadores; (3) respuesta HTTP 400 ante total mínimo negativo; (4) acceso del rol CONSULTA a la vista JSP; (5) redirección de usuarios anónimos; (6) rechazo de cliente vacío; (7) rechazo de minPedidos=0; y (8) respuesta JSON exitosa para clientes frecuentes.

El patrón de organización es Arrange–Act–Assert: se preparan filas de prueba, se invoca el repositorio o endpoint y se verifican resultados mediante assertions de JUnit y MockMvc. Las pruebas existentes del proyecto base se conservan, pero no se contabilizan como las nuevas pruebas PA3.

Ejecución requerida

Desde la raíz del proyecto, ejecutar **mvnw.cmd clean test** en Windows. Adjuntar una captura de la consola donde se vea el resultado final BUILD SUCCESS y los conteos de pruebas. El entorno de preparación de este archivo no cuenta con Maven disponible ni pudo descargar la distribución del wrapper; por ello, el estado de compilación y ejecución debe verificarse en el equipo local antes de la entrega.

6. Configuración y reproducibilidad

La ejecución normal utiliza MySQL. Se incluye database/Script_BD_usuario_MySql.sql para crear pedidosdb y el usuario de laboratorio. Las credenciales se externalizan mediante DB_URL, DB_USERNAME y DB_PASSWORD. H2 se reserva para pruebas. El proyecto requiere JDK 21, Maven Wrapper, MySQL y, para registrar pedidos, inventario-service disponible en su dirección configurada.

7. Conclusiones

La evolución propuesta mantiene la separación de responsabilidades del proyecto base. Spring Data JPA continúa atendiendo la operación de pedidos, mientras que Spring JDBC se utiliza para las consultas de supervisión exigidas en PA3. El uso de parámetros nombrados reduce el riesgo de inyección SQL y permite reutilizar el RowMapper para los resultados de tipo Pedido.

La consulta de clientes frecuentes aplica agregaciones SQL y entrega una proyección específica, evitando mezclar resultados agregados con la entidad Pedido. La vista JSP y la API JSON comparten el mismo servicio, por lo que las reglas de validación no se duplican en los controladores.

Spring Security protege los reportes para ambos roles autorizados, mantiene el registro de pedidos restringido a OPERADOR y conserva CSRF. Las pruebas nuevas cubren consultas, validaciones y seguridad; sin embargo, la entrega final debe acompañarse de la ejecución real de mvnw clean test y de las cinco evidencias funcionales.

8. Referencias (formato APA)

Apache Software Foundation. (s. f.). *Maven Wrapper*. https://maven.apache.org/wrapper/

Oracle. (s. f.). *Java Platform, Standard Edition Documentation*. https://docs.oracle.com/en/java/

Spring. (s. f.). *Spring Framework Reference: Web MVC*. https://docs.spring.io/spring-framework/reference/web/webmvc.html

Spring. (s. f.). *Spring Framework Reference: JDBC*. https://docs.spring.io/spring-framework/reference/data-access/jdbc.html

Spring. (s. f.). *Spring Security Reference: Authorization*. https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html

Spring. (s. f.). *Spring Security Reference: CSRF Protection*. https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

Spring. (s. f.). *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/reference/

Pendiente de completar antes de entregar

Agregar NRC y nombre del docente en la carátula. Incorporar URL del repositorio personal, rama y hash del commit final. Adjuntar las capturas reales de los cinco escenarios y la salida de mvnw.cmd clean test. El ZIP es una copia de trabajo y no sustituye el repositorio Git individual solicitado por la evaluación.